

Neurix Data Entry Management System.

A full-stack workspace for teams that push a lot of structured data through a small number of hands — and want the paper trail, the analytics, and the guardrails to come for free.

DOCUMENT

Technical Documentation

Version 1.0

AUTHOR

Neurix Team

neurix.ai0@gmail.com

PUBLISHED

August 2026

Living document

What's inside.

A quick map so you can jump to whatever you actually need. Chapters are ordered roughly from "why this exists" down to "how do I run it in production".

01	About This Document	4
02	Executive Summary	5
	What Neurix is, who it's for, and the shape of the codebase in numbers.	
03	System Architecture	7
	Three-tier layout, request flow, container topology.	
04	Tech Stack & Rationale	9
	Every library we picked and the tradeoff we made picking it.	
05	Feature Walkthrough	12
	Authentication, dashboards, tickets, custom fields, reports, chat assistant.	
06	Design System	18
	Palette, typography, spacing, motion, accessibility.	
07	API Reference	22
	Every REST endpoint, grouped by controller.	
08	Database Schema	26
	Thirteen domain entities and how they relate.	
09	Security	28
	JWT flow, password hashing, CORS, rate limiting.	

10	Deployment	30
	Docker Compose in production, environment variables, DB swap.	
11	Development Setup	32
	Local dev without Docker, testing, linting.	
12	User Guides	33
	Task-based walkthroughs — submit a ticket, add a user, build a form, run a report.	
13	Troubleshooting	36
	The most common issues and the exact fixes.	
14	Roadmap & Known Limitations	38
	What's stubbed, what's next, what we're deliberately not building.	
A	Appendix — File Layout & Conventions	39
B	Appendix — Glossary	40

About this document.

What you're reading, who it's for, and the assumptions I've made about the reader.

This is the technical documentation for **Neurix**, a data-entry management system built end-to-end over the last few weeks. It covers the codebase at the version rendered on the cover page — if you're reading a printed copy months from now, please treat any specific line count as a snapshot rather than a promise.

I wrote it for three audiences, in decreasing order of how much I've optimized for them:

1. **The engineer who has to keep the thing running.** Every architectural choice has a stated reason, so when the reason no longer holds you can revisit the choice without archaeology.
2. **The stakeholder who needs to know what Neurix does without opening a Java file.** Chapters 2 and 5 are structured so you can read them alone and get the shape of the product.
3. **Future me.** If I come back in six months and I've forgotten why the JWT expiry is 24 hours and not 8, the answer should be in here.

Reading conventions

Code paths and identifiers appear like this:

`backend/src/main/java/com/dataentry/service/ChatService.java`. Terminal commands look like this:

```
docker compose up --build
```

Callouts flag context that goes beyond the surrounding paragraph:

 Neutral notes look like this. Background context, non-obvious behavior, that kind of thing.

 Green callouts are practical tips — usually a small habit that saves time.

 Amber callouts flag something you can actually break — configuration that's easy to get wrong or a foot-gun in the code.

What I've deliberately left out

This document doesn't try to be an API tutorial or a Spring Boot / React primer. If you've never seen a JPA entity or a React hook before, some sections will read as reference material rather than teaching. I've linked mental models where I could, but the goal is *documentation of this system*, not a general course.

I've also skipped over the standard boilerplate — `package.json` scripts you can read yourself, `@Autowired` plumbing that looks the same in every Spring project. Where a decision was interesting I've written about it; where it was a default I've stayed quiet.

Executive summary.

What Neurix is, why it exists, and the shape of the codebase in numbers.

What is Neurix?

Neurix is a **data-entry management system**. Think of a call center or a claims-processing team: a group of people who spend their day filling in structured forms, and a smaller group who need to see what got filed, how quickly, and by whom. Neurix is the software those two groups share.

Concretely, it does five things:

- Turns free-form paperwork into **structured tickets** that can be counted and sliced.
- Lets admins **compose the intake form dynamically** without a code change — new fields are added from the UI.
- Organizes work by **department, subcategory, and project**, so you can filter and report on any of those cuts.
- Gives each user a **personal dashboard** and the admin an **organization-wide dashboard**.
- Includes a **rule-based chat assistant** that navigates the app on demand (and is wired to swap in a real LLM whenever the budget arrives).

Who uses it, and how

Neurix has two roles: **Admin** and **User** (a "data-entry agent" in the UI). They share the same login screen and the same navigation shell, but their home pages, their sidebars, and their permissions diverge.

The admin's day

An admin lands on `/admin`, which is a five-KPI dashboard: total tickets, tickets today, users, departments, and a small trend chart. From there they typically go one of four places:

1. **Users** — to add a new agent, change someone's role, or check who's active. Each user's row links to a per-user *activity page* that shows what they've submitted and when.
2. **Departments & Subcategories** — the taxonomy that every ticket has to slot into.
3. **Custom Fields** — the interesting one. This is where the admin adds a new field to the submission form. Toggle a field on and every agent sees it the next time they load the form.
4. **Reports** — analytics: per-department volume, per-user performance, per-status breakdowns, time trends.

The agent's day

An agent lands on `/dashboard`, which is their *own* dashboard: personal KPIs, a streak, a status donut, a recent-activity feed. Then they go to `/submit`, fill in the form (including any custom fields the admin has defined), and optionally run the *AI content check* before submitting. Their tickets show up under `/my-tickets` with status, feedback, and history.



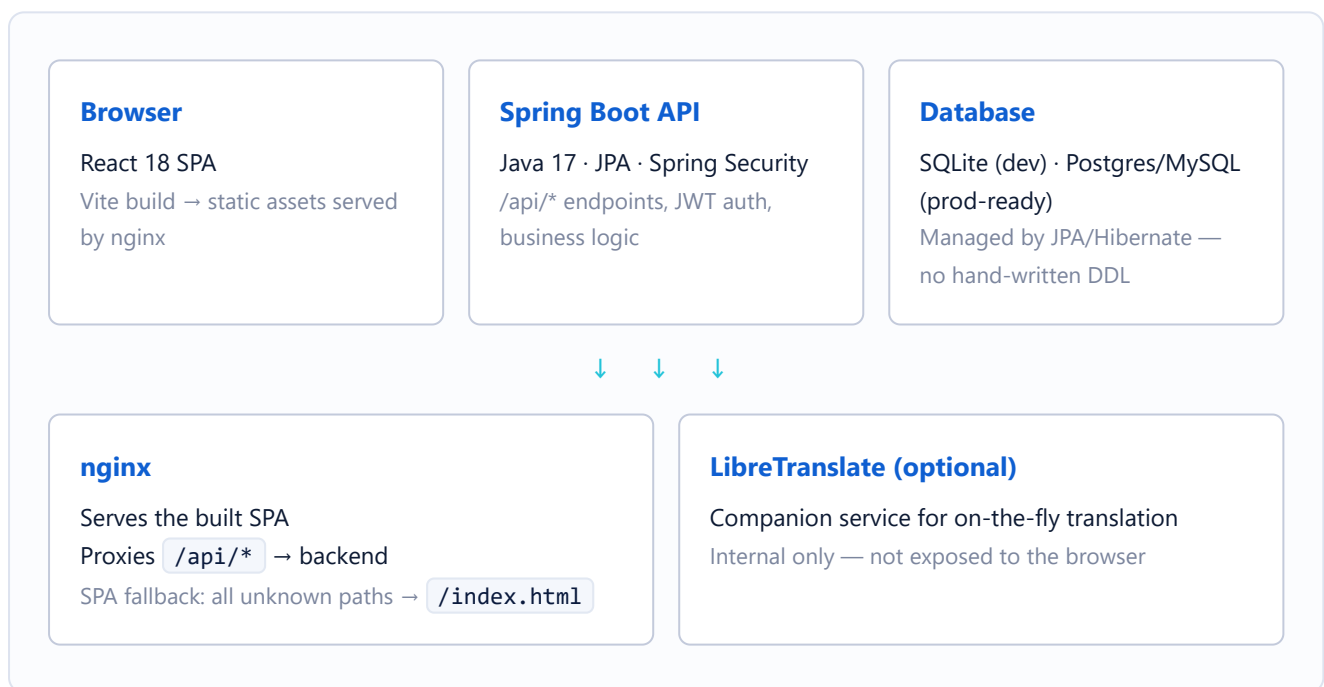
Admins can access everything an agent can — the reverse is not true. That asymmetry is enforced in Spring Security (`hasRole('ADMIN')` vs `hasAnyRole('USER', 'ADMIN')`) and in the React router (`<ProtectedRoute roles={['ADMIN']}>`). Both matter — the frontend guard is cosmetic, the backend guard is the actual authorization boundary.

System architecture.

Three tiers, no surprises. Where each tier lives, what it owns, and how they talk.

The 30-second version

Neurix is a fairly boring three-tier web app: a React SPA in the browser, a Spring Boot REST API on the server, and a SQL database behind that. In production the frontend and backend each run in their own Docker container, with nginx fronting the SPA and proxying `/api/*` back to Spring. There's a third container for LibreTranslate (used by the localization / translation service), running quietly in the background.



Container topology in production

The `docker-compose.yml` at the repo root brings up three services:

Container	Image	Exposed port	Role
dems-frontend	Built from <code>frontend/Dockerfile</code> (nginx-alpine base)	8082 → 80	Serves the SPA + proxies API
dems-backend	Built from <code>backend/Dockerfile</code> (JRE 17 base)	8083 → 8080	The Spring Boot REST server
dems-libretranslate	<code>libretranslate/libretranslate:latest</code>	internal (5000)	On-demand text translation

! The nginx config in `frontend/nginx.conf` does two things you need to know about: (a) it proxies `/api/*` to the backend service by its *container name*, so the frontend Docker image is useless without Compose, and (b) it falls back to `index.html` for any request that doesn't match a real file. That fallback is what allows client-side routing to work on refresh — but it also means a missing asset returns a 200 with HTML, not a proper 404. Watch for that when you're debugging why an image isn't loading.

The request lifecycle

Take a typical action — an agent submitting a ticket — and follow it through the stack:

1. The agent clicks *Submit* in the browser. React calls `api.post('/user/tickets', payload)` from `src/api/tickets.ts`.
2. The axios interceptor in `src/api/client.ts` attaches the JWT (from a cookie, falling back to an in-memory token for backwards compatibility) and adds an `Accept-Language` header from `localStorage`.
3. The request hits nginx at `/api/user/tickets`, which rewrites it and forwards to `dems-backend:8080/api/user/tickets`.
4. Spring's security filter chain (`SecurityConfig.java`) matches the URL against the rules: `/api/user/**` requires `ROLE_USER` or `ROLE_ADMIN`. If the JWT is valid, the authentication object gets populated; otherwise 403 is returned.
5. `TicketController` receives the DTO, hands it to `TicketService`, which validates business rules and persists via `TicketRepository`.
6. Hibernate emits SQL against SQLite (or Postgres, if configured), the entity is saved, and the ID is returned.
7. `TicketService` also writes an audit-log entry via `AuditService`. The audit table is append-only — nothing else in the app writes to it, and no admin UI lets you delete rows.

8. The controller returns the ticket DTO; nginx passes the JSON back to the browser; React updates state and shows a success toast.

Every write path in the app follows this same skeleton. Once you've internalized the six pieces — *API wrapper* → *interceptor* → *nginx* → *security filter* → *controller* → *service* → *repository* — the whole backend reads the same way.

Tech stack & rationale.

Every non-trivial library and the tradeoff behind picking it. If a library isn't listed here it's because it was a default that didn't earn a discussion.

Backend

Java 17 + Spring Boot 3.3.5

Spring Boot is the boring, default choice for a Java REST API, which is exactly why I picked it. Everything a small team needs — HTTP, validation, DI, security, JPA — comes wired together, and I never had to fight the framework to get standard behavior. Java 17 is the current LTS at the time of writing; the codebase uses records for DTOs and switch expressions where they help readability, but nothing exotic.

The concrete Spring starters in play:

Starter	What it gives us
<code>spring-boot-starter-web</code>	Tomcat, MVC, JSON via Jackson
<code>spring-boot-starter-data-jpa</code>	Hibernate + repository abstractions
<code>spring-boot-starter-security</code>	Filter chain, method-level authorization
<code>spring-boot-starter-validation</code>	Bean-validation on DTOs (<code>@Valid</code> , <code>@NotBlank</code> , etc.)

SQLite by default, Postgres/MySQL on demand

SQLite makes local development a single command with zero prerequisites: `docker compose up` and you're running. The database file lives in a named volume (`dems-data`), so it survives container restarts but wipes with `docker compose down -v`. This is the right tradeoff for a demo / small team — SQLite handles the write volume of a data-entry system without breaking a sweat.

Swapping to Postgres or MySQL is a two-line change in `application.yml` plus a JDBC driver swap in `pom.xml`. Every entity uses plain JPA — no SQLite-specific tricks, no native queries with dialect quirks. See Chapter 10 for the exact incantation.

! SQLite's write serialization is fine at low concurrency but will bite you above a few dozen simultaneous writers. If you're planning to onboard hundreds of concurrent agents, do the Postgres swap first — it's cheap up front, painful under load.

JWT auth via JJWT 0.12.x

The auth flow is intentionally standard: `POST /api/auth/login` exchanges username + password for a signed HS512 JWT. The token is set as an `httpOnly` cookie *and* returned in the response body, because early versions stored it in `localStorage` and we still have the migration path.

Passwords are hashed with BCrypt (via `spring-security-crypto`) at a cost factor of 10 — the framework default. This is *not* the strongest possible setting, but it's the standard tradeoff between hash time (~100ms) and login latency at scale.

Rate limiting: in-memory and DB-backed variants

Login attempts are throttled by IP + username via `LoginRateLimiter`, which has two implementations: `InMemoryLoginRateLimiter` for single-node deployments and `DatabaseLoginRateLimiter` when you're running multiple backend instances behind a load balancer. The in-memory one is the default; swap in the DB version by changing which bean is qualified as `@Primary`.

Frontend

React 18.3.1 + TypeScript 5.6.3 + Vite 5.4.10

React + TypeScript is the least controversial UI stack a new project can pick in 2026. Vite as the bundler gives you sub-second dev-server startup and, in this project, a full production build in a couple of seconds. I don't have strong feelings about Vite vs. its competitors — it just works, and I've never had to think about it after the initial `npm create`.

There are no state-management libraries — the app is small enough that React Context (for auth, theme, and language) plus local `useState` covers everything.

React Router 6.27.0

Standard nested routes, with two layers of protection: `<ProtectedRoute>` to bounce unauthenticated users to `/login`, and a `roles={['ADMIN']}` prop for admin-only routes. See `src/App.tsx` for the whole routing tree.

axios 1.7.7

One axios instance in `src/api/client.ts`, one interceptor for auth + language headers, and a thin wrapper file per resource (`src/api/tickets.ts`, `chat.ts`, etc.). The wrappers deliberately keep the axios types out of the consuming components — every hook and component consumes typed response objects, not `AxiosResponse<...>`.

Hand-crafted design system (no UI library)

Instead of a component library — Material, Chakra, shadcn, etc. — the whole UI is a hand-written CSS design system in one file: `frontend/src/styles/global.css`. This is a real tradeoff, not a mistake:

- **Pro:** Zero framework churn, no bundle-size hit for components we don't use, complete control over motion and color, and the whole system fits in one file you can grep.
- **Pro:** The palette matches the Neurix identity exactly, because I picked every hex.
- **Con:** You give up the accessibility polish that comes with a well-maintained kit — every focus ring, every ARIA attribute is my problem.
- **Con:** New components take longer to build than dropping in `<Combobox>` from someone else's library.

For a system this size, with a strong brand identity, the tradeoff was worth it. For a larger app or a team of five, I'd probably reach for shadcn.

Infrastructure

Piece	What it is	Why it's here
Docker Compose	Orchestrates the 3-container setup	One-command bring-up, dev/prod parity
nginx (alpine)	Serves the SPA and proxies <code>/api/*</code>	Standard SPA hosting; no separate API gateway needed
LibreTranslate	Self-hosted translation service	Optional dependency for the translation feature — no vendor API keys required
OpenJDK 17 JRE	Base image for the backend container	Small footprint, LTS support

Notably absent

Things that don't exist in this stack, and why:

- **No Redis.** The rate limiter and audit log both use the primary database. If you outgrow that, Redis is the next stop.

- **No message queue.** All writes are synchronous. If we grow into background jobs (PDF exports, batch imports), the natural next step is an in-process queue or a small worker service.
- **No frontend state library (Redux, Zustand, Jotai).** The app doesn't need one yet. React Context handles auth, theme, and language cleanly.
- **No GraphQL.** The REST surface is small (12 controllers) and the client uses each endpoint from one or two call sites. GraphQL would be pure overhead here.

Feature walkthrough.

Each major capability, what it does, where the code lives, and any gotcha worth flagging.

Authentication & sessions

Backend: `controller/AuthController.java`, `service/AuthService.java`. *Frontend:* `src/context/AuthContext.tsx`, `src/pages/LoginPage.tsx`.

Users land on `/login`, submit a username + password, and get back a JWT that's set as an `httpOnly` cookie (server-side) and cached in an axios interceptor (client-side). The token is HS512-signed, expires in 24 hours, and encodes three claims: `sub` (username), `uid` (user ID), and `role` (`ADMIN` or `USER`).

On refresh, the frontend calls `GET /api/auth/me` — if the cookie is still valid, the backend replies with the user object; otherwise a 401 sends them back to `/login`. This is why the login page shows for a beat during boot: React is waiting on the auth resolution before deciding where to route you.



The default admin credentials (`admin / admin123`) are seeded on first run by `DataSeeder`. Change them the moment you deploy anywhere real. The `JWT_SECRET` env var also has a hard-coded default; production deployments should override it.

Admin dashboard

Backend: `controller/DashboardController.java`, `service/DashboardService.java`. *Frontend:* `src/pages/admin/AdminDashboardPage.tsx`.

A single endpoint (`GET /api/admin/stats`) returns everything the admin dashboard needs: total tickets, tickets today, active users, department count, a hero-metric card, and a small 7-day trend. Aggregation happens in SQL via Spring Data repository methods — nothing is computed in Java. The frontend renders five stat cards, a status donut, and a trend chart, all styled to match the Neurix palette.

User management

Backend: `controller/AdminUserController.java`, `service/UserService.java`. *Frontend:* `src/pages/admin/AdminUsersPage.tsx`, `AdminUserActivityPage.tsx`.

Admins can create, edit, disable, and delete users. Passwords are set at creation and can be reset from the row-actions menu. Each user row links to a *per-user activity page* that shows their submitted tickets, average processing time, and current streak — the same data the user themselves sees on their own dashboard, but visible to the admin.

Departments, subcategories, projects

Backend: `DepartmentController`, `SubcategoryController`, `ProjectController` + matching services.

Frontend: the three `Admin{Department,Subcategory,Project}sPage.tsx` files.

The taxonomy is a two-level hierarchy: *Departments* contain *Subcategories*, and every ticket has a required `departmentId` + optional `subcategoryId`. Projects are an orthogonal grouping — a project can span departments and is used mainly for reporting.

Each department gets one of six color slots (`--dept-color-1` through `--dept-color-6`) that show up as a top-border strip on its card in the admin UI. The color assignment is deterministic and lives in the frontend — the backend just stores the department name and metadata.

Custom form fields (the dynamic form)

Backend: `controller/CustomFieldController.java`, `service/CustomFieldService.java`, `model/CustomField.java`, `model/FieldType.java`. *Frontend:* `src/pages/admin/AdminFieldsPage.tsx`, `src/pages/user/submit/CustomFieldsSection.tsx`, `DynamicField.tsx`.

This is the feature that earned the app its keep. An admin can go to *Custom Fields*, add a field (text / number / date / dropdown / textarea, etc.), and every agent immediately sees it on the submission form on their next page load. No code change, no deploy.

The trick is that field *values* aren't stored as columns on the `tickets` table — they're stored as rows in a `ticket_field_values` table, keyed by `(ticket_id, field_id)`. This is the EAV pattern, and it's the right shape for this kind of feature. It costs you the ability to write typed SQL against the values (they're all stored as strings), but it buys you unlimited schema flexibility from the UI.

i Fields are soft-toggleable: disabling a field hides it on the submission form but doesn't delete its historical values. This is important for reporting — you can turn a field off without losing the data agents already entered against it.

Ticket submission workflow

Backend: `controller/TicketController.java`, `service/TicketService.java`, `model/Ticket.java`, `TicketFieldValue.java`, `TicketStatus.java`. *Frontend:* `src/pages/user/SubmitTicketPage.tsx` and the

`submit/` subdirectory.

A ticket has a fixed core (*title, department, subcategory, content, status, created-at, submitted-by*) and a variable body (*custom-field values*). Statuses are: `IN_PROGRESS`, `REVIEW`, `COMPLETED`. There's also an implicit `OVERDUE` that's derived from the submitted-at timestamp — not a real DB status.

Agents can attach documents. Uploads go through `UploadQuotaService` (which enforces per-user daily quotas) and, for PDFs, through `PdfExtractionService` or `PdfOcrService` depending on whether the PDF is a text PDF or a scan. The extracted text pre-fills the content field.

AI content check

Backend: `controller/AiCheckController.java`, `service/AiCheckService.java`. *Frontend:* `src/pages/user/submit/AiCheckDialog.tsx`.

Before submitting, an agent can hit *Check with AI*. The current implementation is a deterministic cleanup — normalization, whitespace, common typos — not a real LLM call. The API contract is deliberately shaped like a real LLM would return: `{ original, corrected, notes: string[] }`. Swapping in Claude or GPT is a single method change in `AiCheckService.check()`.

Reports & analytics

Backend: shares the `DashboardService` for its aggregations. *Frontend:* `src/pages/admin/AdminReportsPage.tsx`.

The reports page slices ticket data along three axes: *by department, by user, by status*. Time ranges are toggleable (today / this week / this month / all time). Every chart is SVG rendered by hand — no chart library — which keeps the bundle small and the styling on-brand.

Neurix Assistant (chat)

Backend: `controller/ChatController.java`, `service/ChatService.java`. *Frontend:* `src/components/chat/ChatWidget.tsx`, `src/api/chat.ts`.

A floating circular button lives in the bottom-right on every authenticated page. Clicking it opens a chat panel. The user types a question in Arabic or English; the backend classifies the intent and returns a reply plus (optionally) navigation *actions* — buttons like "Open Users →" that the frontend hands to React Router.

Intents are handled with keyword matching over a bilingual knowledge base. Categories:

- **Small talk** — greetings, thanks, farewells.
- **Site info** — "what does this do", "الموقع بيعمل ايه".

- **Help** — capability listing.
- **Navigation** — every page has a keyword set in both languages, plus a role gate (admin-only pages are filtered out for agents).
- **Fallback** — quick-links to the three most common destinations for the current role.

Conversation history is persisted in `localStorage` (last 40 messages), so the chat picks up where you left off across page refreshes. When the LLM budget arrives, `ChatService.reply()` is the one method that changes.

Internationalization (English + Arabic, RTL)

Frontend: `src/i18n/index.tsx`, `en.ts`, `ar.ts`.

Every string in the UI goes through `t('key.name')`. The dictionaries are two TypeScript files with a nested-object shape; the `lookup()` helper walks the dot-path at runtime. Language toggle lives in the preferences drawer and persists to `localStorage` (`dems.lang`).

Arabic flips the whole app to RTL. This is *not* just `dir="rtl"` on the root — every CSS rule that uses `margin-left` or `padding-right` has been rewritten to use logical properties (`margin-inline-start`, `padding-inline-end`). The design system's spacing all mirrors correctly.

Dark mode

Theme toggle is next to the language toggle. The dark palette is a full re-derivation of the light palette — every token has a dark counterpart in `global.css`, not just a naive color-invert. Both modes use the Neurix identity; dark just lifts the primary blue and cyan for contrast against the deep-navy surfaces.

Design system.

The Neurix design system in three layers: **Foundations** (the raw tokens), **Components** (how those tokens combine), and **Patterns** (how components combine into recurring solutions). Every visual example on the following pages is rendered live in the same CSS the app ships with — what you see here is what you get in the browser.

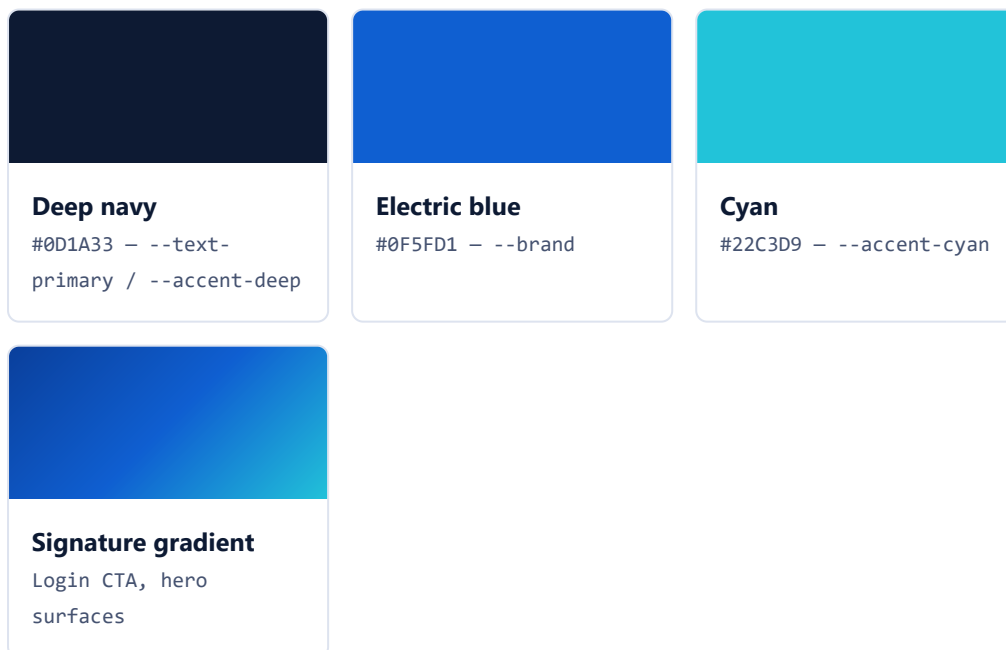
Part 1 • Foundations

The tokens below are the smallest units of the system. Everything in the app — every button, every card, every focus ring — is composed from these.

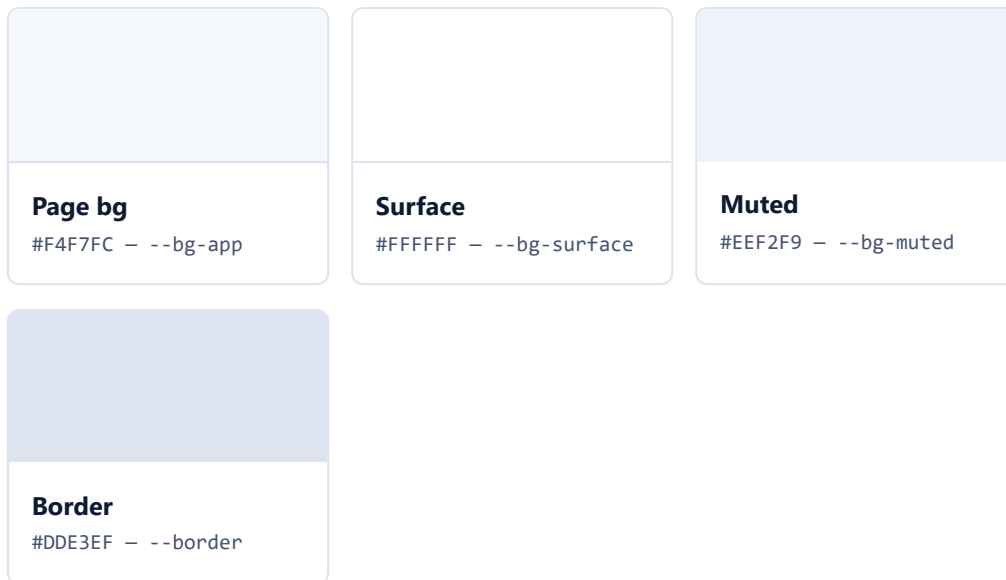
Palette

Sampled directly from the Neurix logo: deep navy for anchoring text, electric blue as the primary action color, cyan as the highlight. Every color in the app is a CSS variable defined once at the top of `global.css`. No hex codes are hardcoded in components — the one exception is the login page's animated background, which sits outside the theme system on purpose.

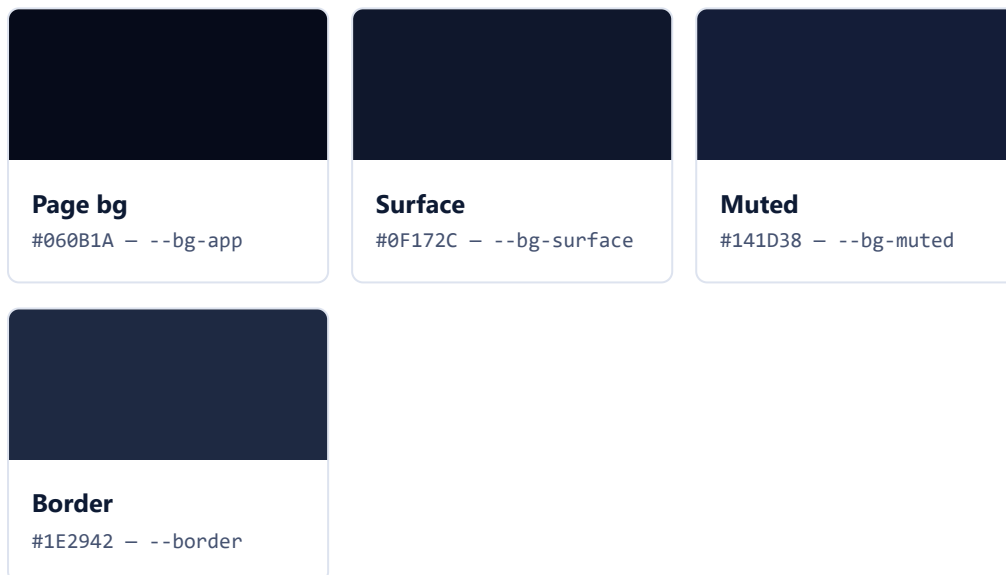
BRAND



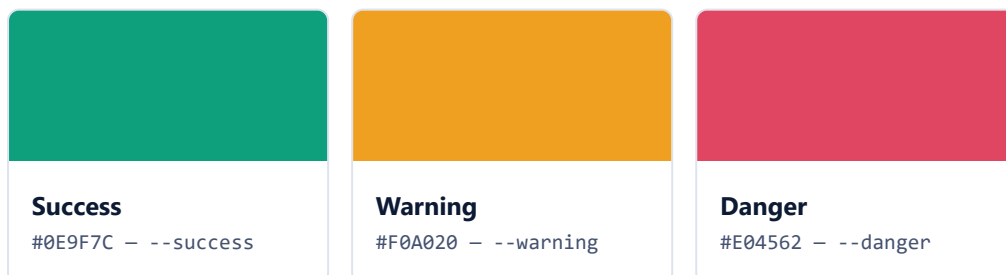
NEUTRALS — LIGHT



NEUTRALS — DARK



SEMANTIC STATUS



Violet (dept)
#7C56D9 — --dept-color-3

i Every hue has a "soft" pair. For every status color there's a matching soft background: `--danger-soft`, `--success-soft`, etc. Rule: use the strong color for the border or the icon; use the soft color for the fill. That way status pills, alerts, and badges all read at a glance without dominating the surrounding page.

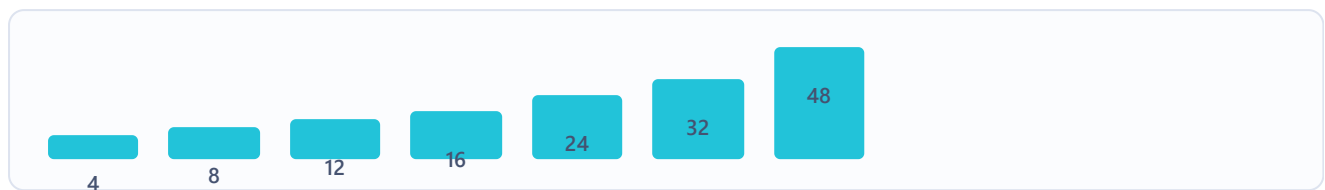
Typography

One family (*Inter*) at five weights (400, 500, 600, 700, 800), with a monospace fallback (*JetBrains Mono*) for code, credentials, and numeric data. Arabic falls back to *Segoe UI Arabic* / *Tahoma* / *Cairo*. The ordering matters — Segoe renders Arabic beautifully on Windows and Cairo covers everywhere else.

HERO H1	2rem • 800 • -0.025em	Manage data entry with confidence
PAGE H2	1.6rem • 700 • -0.015em	Team Roster
PANEL H3	1.25rem • 700	Recent Activity
BODY	0.95rem • 400 • 1.55	One workspace to organize your data entry team, track tasks, and monitor performance in real time.
SMALL	0.875rem • 400	Sign in to the Data Entry Management System
LABEL	0.75rem • 600 • caps • +0.08em	USERNAME
MONO	JetBrains Mono • 0.9em	TX-9K42 • \$14,720

Spacing scale

Everything sits on an 8-pixel grid. You won't find a 7px or 15px anywhere in the CSS — every padding, gap, and margin picks from this scale.



Border radius

Token	Value	Used for
<code>--radius-sm</code>	6px	Chips, tags, inline code, tiny buttons
<code>--radius</code>	10px	Inputs, alerts, side-panel headers
<code>--radius-lg</code>	16px	Cards, modals, table wrappers, dashboard panels
<code>--radius-pill</code>	999px	Every button and every status pill

Elevation

Three shadow steps plus a "flat" state. Neurix uses elevation to signal focus, not to decorate — flat surfaces are the default; shadow appears when something floats above the page (modal, popover, hovered card).

none flat surface	--shadow-sm cards at rest	--shadow hover state	--shadow-lg modal / drawer
-----------------------------	-------------------------------------	--------------------------------	--------------------------------------



No colored glow shadows. The only chromatic box-shadow in the whole system is the focus ring (cyan) and the primary CTA's ambient lift on the login page. Everything else is a neutral, near-black shadow. Colored glows read as "AI cool" and get old fast.

Motion tokens

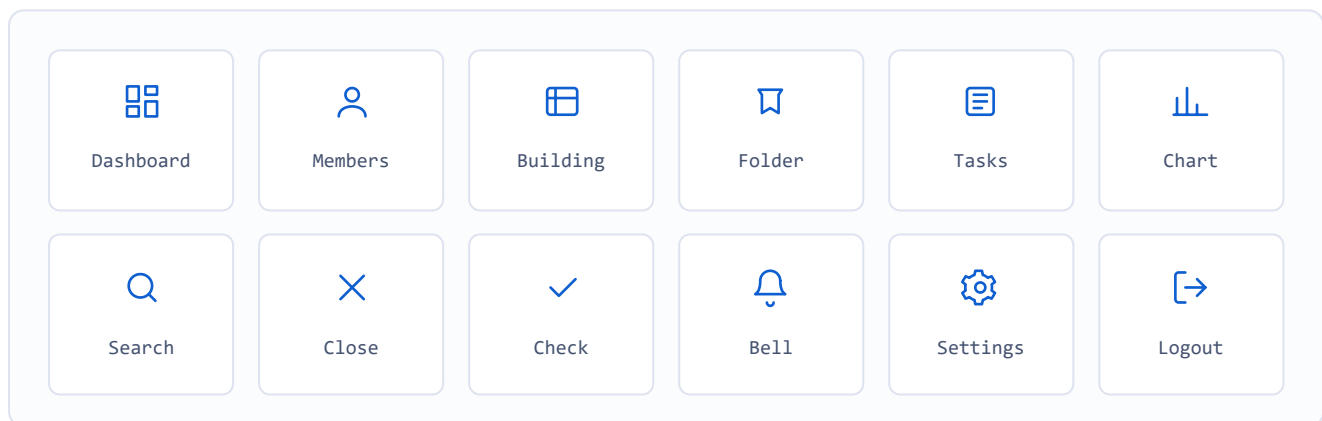
Motion is deliberately restrained. Cards don't lift on hover — they get a slightly stronger border and a wider shadow, which is calmer than a transform. For a tool people use eight hours a day, aggressive motion becomes fatigue.

150ms quick, ease <i>Button hover, input border color</i>	200ms standard, ease <i>Card border/shadow, chip states</i>	300ms deliberate, cubic-bezier(.2,.7,.2,1) <i>Panel / drawer entrance</i>
500ms flowy, ease <i>Bars, progress transitions</i>	700ms choreographed, cubic-bezier(.2,.7,.2,1) <i>Auth entrance sequence</i>	rAF frame-driven <i>Login canvas particle network, chat typing dots</i>

Every animation is wrapped in a `prefers-reduced-motion` media query: the login canvas disappears entirely; the chat pulse ring stops; entrance animations reduce to instant.

Icon system

Icons are hand-drawn 24×24 SVG paths, one per glyph, defined in `src/components/Icons.tsx`. Stroke-based, 1.6px stroke width, rounded joins. They inherit `currentColor`, so wherever they sit their color matches the surrounding text. No icon library, no external font.

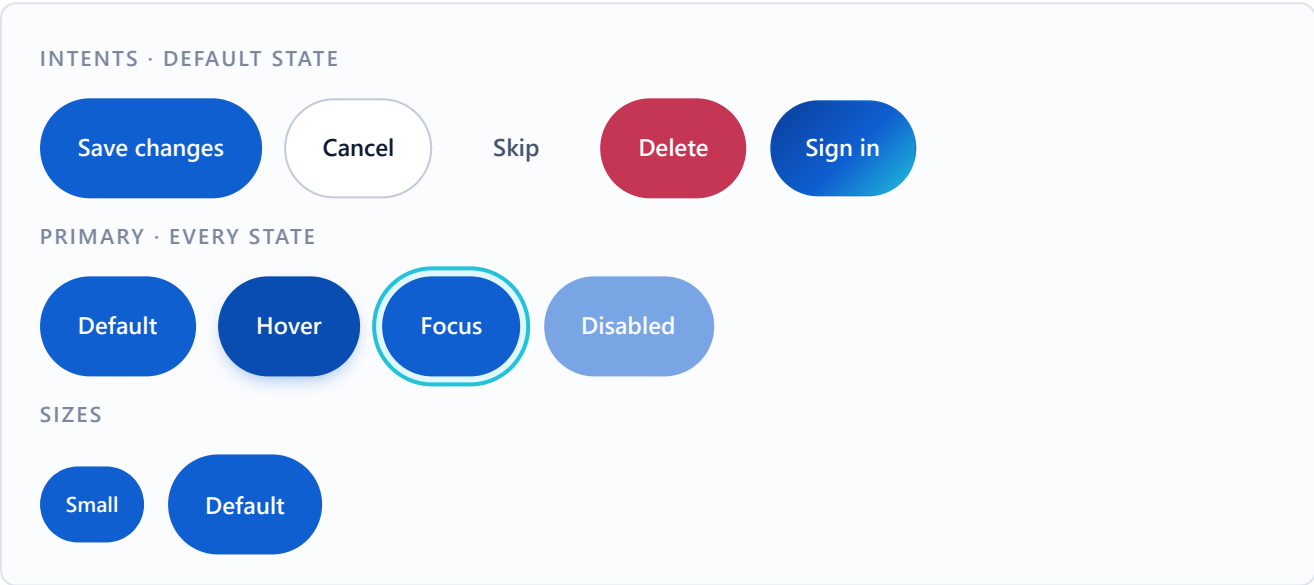


Part 2 · Components

The Neurix component library. Each spec below shows the default rendering plus every meaningful state — you can point a designer at these pages and they'll know exactly what the shipped app looks like.

Buttons

All buttons are pills (`border-radius: 999px`), min-height 42px, weight 600. Four intents: *primary*, *secondary*, *ghost*, *danger*. Plus a special *gradient* variant reserved for the login CTA.



Intent	When to use it
Primary	The single most important action on a screen. Never two on one screen.
Secondary	Alternative actions of equal or lesser weight. "Cancel", "Save as draft".
Ghost	Tertiary actions. Row-actions in tables, "Skip" in dialogs.
Danger	Destructive actions. Confirm-delete dialogs, permanent removals.
Gradient	Reserved for the login CTA. Don't use elsewhere — its power comes from being rare.

Form fields

Inputs have min-height 44px, radius 10px, always paired with an uppercase-tracked label above. Focus ring is cyan (matches the login-panel accent). Error state uses the danger token with a matching soft halo.

TEXT INPUT · EVERY STATE

DEFAULT

your username

FOCUS

agent1|

ERROR

a|

Username must be at least 3 characters.

DISABLED

read-only value

Cards

Radius 16px, 24px inner padding, shadow-sm at rest, shadow (larger) on hover. Cards never move on hover — the elevation change is enough to communicate interactivity without the jitter of a transform.

Total tickets today

248

▲ 12% vs. yesterday

Engineering

14 active tickets · 3 in review

Status pills

Pills carry ticket status. Text + soft background + a leading dot in `currentColor`. The dot is what your eye lands on when scanning a table of 200 rows.

• In progress

• Review

• Completed

• Overdue

Alerts

Full-width strip with a matching border and soft fill. Never with an icon — the color is enough, and the copy tells the user what happened.

A department was created successfully.

Ticket TX-9K42 was submitted.

You have 3 tickets waiting on review.

Login failed — check your username and password.

Avatars

Circular, initials-only. Six deterministic hue slots derived from the user's ID — so the same person always has the same color, but no color is "special". Three sizes.

HUES 1 – 6 · SIZE SM / MD / LG

AK

AK

AK

MO

SH

YS

RA

NM

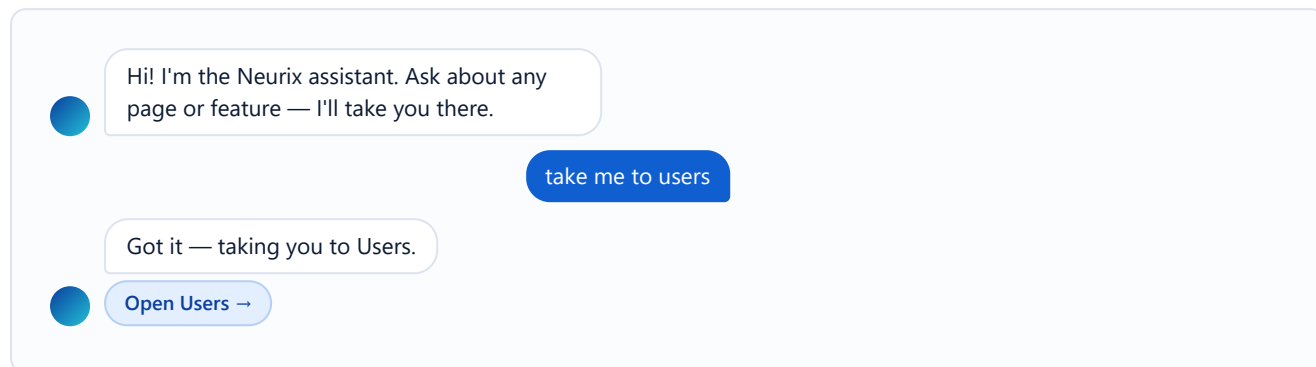
Tables

Uppercase-tracked headers on a subtle sunken row, alternating row backgrounds for scannability, row hover for the "I'm looking at this one" feedback. No zebra-stripes below a header — the header alone is enough separation.

TICKET	DEPARTMENT	STATUS	SUBMITTED
TX-9K42	Engineering	• In progress	2m ago
TX-9K41	Operations	• Review	18m ago
TX-9K38	Engineering	• Completed	1h ago
TX-9K35	Finance	• Overdue	3h ago

Chat bubbles & action chips

The Neurix Assistant's message list. Bot messages get a light surface with a small bottom-left corner tuck; user messages get the brand blue with the opposite corner tuck. Action chips sit under the bot's reply and are the primary way the assistant hands you off to another page.

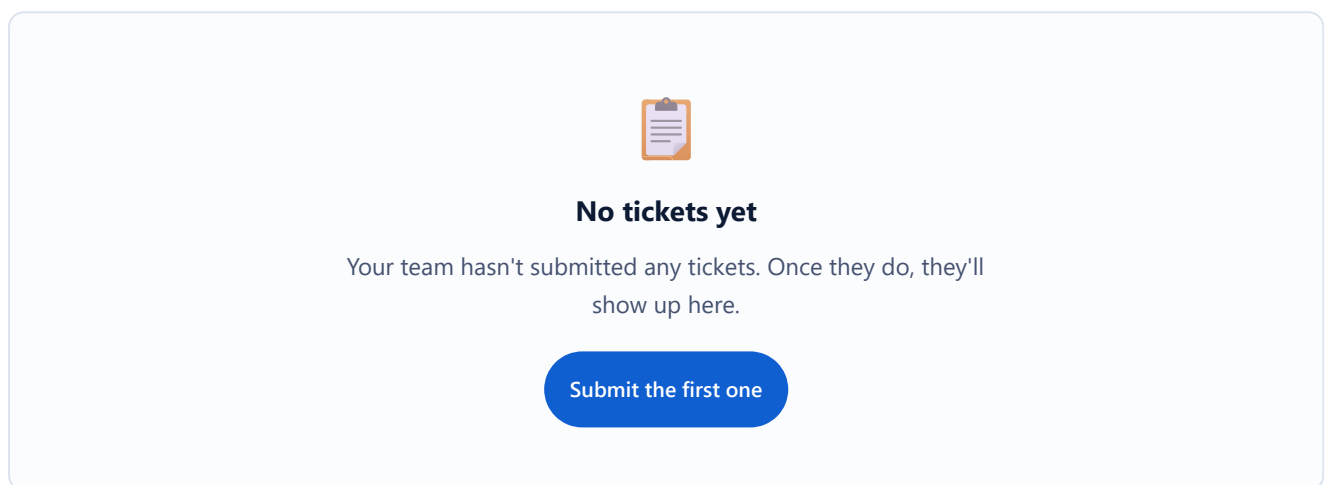


Part 3 · Patterns

Higher-order rules for how components combine, and how the app should behave in ambiguous moments — empty states, errors, first-run, silence.

Empty states

Never leave an empty table or an empty page just blank. Every list has a designed empty state with three parts: a soft icon, a one-line title, and a one-line description of what to do next. If the user can act, put a primary button. If they can't, don't — false CTAs are worse than none.



Loading states

Two patterns. For quick loads (< 300ms) show nothing — snapping in is calmer than a flash of spinner. For anything longer, show a skeleton (a soft gray outline of the shape the content will take), not a spinner. The one exception is the sign-in button, which shows an inline spinner because the button itself is the loading target.

Error states

Three tiers:

1. **Inline field error.** Red border on the input + a one-line message below. Always in the copy the user needs to read to fix it, never a code.
2. **Toast / alert strip.** For actions that failed. Includes a "Retry" affordance when the action is safe to repeat.
3. **Full-page error.** For catastrophic failures. Includes the incident ID (so support can find the logs) and a "Go home" button.

Focus rings — the one place we allow chromatic shadow

Every interactive element carries a visible `:focus-visible` ring, drawn with the cyan accent (`0 0 0 3px var(--accent-cyan-soft), 0 0 0 5px var(--accent-cyan)`). This is the only place in the whole system where a colored halo appears — because focus rings are a WCAG requirement and the brand accent is the right hue to signal "your keyboard is here".

Copy voice

- **Short, human, direct.** "Save changes", not "Persist your modifications". "Something went wrong" is better than "An error has occurred".
- **You, not the user.** "You have 3 tickets" > "The user has 3 tickets".
- **Present tense.** "Ticket submitted" > "Ticket has been submitted".
- **Never blame the user.** "We couldn't reach the server" > "You entered an invalid value".
- **No exclamations, no emoji in state copy.** The chat assistant is the one place emoji can appear — its persona is friendly. Everywhere else stays quiet.

Do & Don't

✓ Do

- Use one primary button per screen.
- Use soft backgrounds + strong text for status.
- Match the border-radius to the element's role (pill for buttons, radius-lg for cards).
- Ship a designed empty state for every list.
- Wrap every animation in `prefers-reduced-motion`.

✗ Don't

- Use gradient text on headings — it reads decorative.
- Use colored glow shadows outside the focus ring.
- Add exclamation marks or promotional emoji in UI copy.
- Add hover transforms to rows or cards (jitter on scroll).
- Hardcode a hex — extend the tokens if a new hue is needed.

Accessibility

- Every button has a visible `:focus-visible` ring, drawn with the cyan accent.
- Form fields are always labeled — either with a visible `<label>` or an `aria-label`.
- Icon-only buttons carry an `aria-label`; loading states use `aria-busy` and screen-reader-only spinner text.
- Color contrast targets WCAG AA at minimum. The status pill text/background pairs and the primary button text on the electric-blue background were both specifically tuned.
- RTL isn't a bolt-on — it uses CSS logical properties throughout (`margin-inline-start`, `padding-inline-end`), so the whole design mirrors correctly without per-page overrides.

API reference.

Every REST endpoint the backend exposes, grouped by controller. All paths are prefixed with `/api`.

Authentication

Method	Path	Auth	Purpose
POST	/auth/login	public	Exchange credentials for a JWT
GET	/auth/me	any	Current user object (uses the auth cookie)
POST	/auth/logout	any	Clear the auth cookie

Admin dashboard & users

Method	Path	Auth	Purpose
GET	/admin/stats	ADMIN	Everything the admin dashboard needs, in one call
GET	/admin/users	ADMIN	List all users (paginated)
POST	/admin/users	ADMIN	Create a user
PATCH	/admin/users/{id}	ADMIN	Update role / display name / active flag
DELETE	/admin/users/{id}	ADMIN	Delete (soft-delete is on the roadmap)
GET	/admin/users/{id}/activity	ADMIN	Per-user KPIs and recent tickets

Departments, subcategories, projects

Method	Path	Auth	Purpose
GET	<code>/admin/departments</code> · <code>/departments</code>	ADMIN · any	Full list (admin) · active only (user form)
POST	<code>/admin/departments</code>	ADMIN	Create a department
PATCH	<code>/admin/departments/{id}</code>	ADMIN	Update name / description / active
DELETE	<code>/admin/departments/{id}</code>	ADMIN	Delete
Subcategories and projects follow the same CRUD pattern under <code>/admin/subcategories</code> and <code>/admin/projects</code> .			

Custom fields

Method	Path	Auth	Purpose
GET	<code>/admin/fields</code>	ADMIN	All fields, including inactive
GET	<code>/fields</code>	any	Active fields only — what the submission form renders
POST	<code>/admin/fields</code>	ADMIN	Add a new field
PATCH	<code>/admin/fields/{id}</code>	ADMIN	Rename, retype, toggle active, reorder
DELETE	<code>/admin/fields/{id}</code>	ADMIN	Delete (also removes historical values)

Tickets

Method	Path	Auth	Purpose
POST	<code>/user/tickets</code>	any	Submit a new ticket
GET	<code>/user/tickets?page=&size=</code>	any	The current user's own tickets
GET	<code>/admin/tickets?page=&size=</code>	ADMIN	Every ticket, with filters
GET	<code>/tickets/{id}</code>	any (own) · ADMIN (any)	Single ticket, with all field values
DELETE	<code>/admin/tickets/{id}</code>	ADMIN	Delete a ticket

AI & Chat

Method	Path	Auth	Purpose
POST	/ai/check	any	Grammar/spelling cleanup — currently deterministic, LLM-ready
POST	/chat	any	Neurix Assistant — bilingual intent classification + navigation actions

Documents & audit

Method	Path	Auth	Purpose
POST	/documents/extract	any	Upload a PDF / image, get back extracted text
GET	/admin/audit-log	ADMIN	Append-only history of every write action
GET	/pdf/{ticketId}	ADMIN or owner	Rendered PDF of a submitted ticket



All non-public endpoints accept the JWT via either the `Authorization: Bearer <token>` header or the `httpOnly` auth cookie. The frontend uses the cookie; direct API consumers can use either.

Database schema.

Thirteen JPA entities. What each one holds and how they connect.

The entities

Entity	Table	Purpose
User	users	Login identity + role (ADMIN / USER) + display metadata
Role	(enum)	Two-valued enum embedded in User
Department	departments	Top level of the taxonomy — every ticket references one
Subcategory	subcategories	Optional second level under a department
Project	projects	Orthogonal grouping used mainly by reports
ProjectStatus	(enum)	Active / On hold / Completed / Cancelled
CustomField	custom_fields	Admin-defined form fields — the "dynamic form" backbone
FieldType	(enum)	TEXT / NUMBER / DATE / DROPDOWN / TEXTAREA / etc.
Ticket	tickets	A submitted entry — the core write-heavy table
TicketFieldValue	ticket_field_values	EAV storage for custom-field values on a ticket
TicketStatus	(enum)	IN_PROGRESS / REVIEW / COMPLETED
AuditLog	audit_log	Append-only history — never updated, never deleted
LoginAttempt	login_attempts	Backs the DB-based rate limiter

Key relationships

The relationships are all standard many-to-one / one-to-many, no join tables:

```
Department 1 — * Subcategory
Department 1 — * Ticket
Subcategory 1 — * Ticket      (nullable – a ticket may have no subcategory)
Project     1 — * Ticket      (nullable)
User        1 — * Ticket      (the submitter)
Ticket      1 — * TicketFieldValue
CustomField 1 — * TicketFieldValue
User        1 — * AuditLog
```

The dynamic form's EAV trick

Instead of adding a database column every time an admin defines a new field, `TicketFieldValue` stores every custom-field value as a row: `(ticket_id, field_id, value_as_string)`. This is the entity-attribute-value pattern, and it's the right choice here because:

- New fields don't require migrations. An admin adds "Order Reference" from the UI and it works on the next page load.
- Fields can be added, renamed, retyped, or disabled without breaking historical rows.
- Reporting joins are simple: `SELECT ticket_id, field_id, value FROM ticket_field_values WHERE field_id = ?`.

What you give up:

- Typed SQL against custom-field values. Everything is stored as `VARCHAR`; numeric or date typing happens at the Java layer.
- Efficient indexed queries on those values. For the current volume this doesn't matter; at millions of rows you'd want a materialized view.

Security.

Authentication, authorization, hashing, rate limiting, and the sharp edges you can accidentally leave exposed.

Password hashing

Passwords are hashed with BCrypt via Spring Security's `PasswordEncoder`, at the default cost factor of 10. This produces a ~60-character hash per password and costs ~100 ms per verification — the standard tradeoff for interactive login. Nothing is ever stored in plain text; even the seeded admin password is hashed on first boot by the seeder.

JWT flow

1. User POSTs to `/api/auth/login` with credentials.
2. `AuthService` looks up the user, verifies the password, and generates a JWT signed with HS512 (the secret comes from `JWT_SECRET`).
3. The token is set as an `httpOnly`, `Secure` cookie and also returned in the response body for older clients.
4. Every subsequent request carries the cookie; a security filter reads it, verifies the signature and expiry, and populates the Spring `Authentication` object.
5. Controllers read the authenticated user via method injection (`Authentication auth`).

Token expiry is 24 hours. There's no refresh-token flow — the tradeoff is that users re-login every 24 hours; the win is that revocation is trivial (just wait) and there's no session store to manage.

Authorization

Two layers, both enforced independently:

1. **Spring Security URL rules** (`SecurityConfig.java`): pattern-matched on the request path. `/api/admin/**` requires `ROLE_ADMIN`. `/api/user/**` requires either role. `/api/**` requires authentication. Everything else is public.
2. **React router guards** (`<ProtectedRoute>`): stop unauthenticated users from ever seeing an admin page's shell, and redirect based on role.

! The frontend guards are UX. The backend guards are security. Never rely on the frontend alone — any user can open dev tools and change their role in localStorage; only the backend's role check matters for authorization.

CORS

The default CORS policy in `application.yml` is `*`, which is fine for local dev and IP-based deployments but should be tightened to your actual domain in production. When you flip it to a specific origin, remember to also set `Access-Control-Allow-Credentials: true` — the frontend uses `withCredentials` to send the auth cookie cross-origin.

Rate limiting

Login attempts are rate-limited per (IP, username) pair. Default limits: 5 failed attempts in 15 minutes triggers a 5-minute cooldown. The implementation lives in `InMemoryLoginRateLimiter`; if you run more than one backend instance, swap in `DatabaseLoginRateLimiter`.

Audit log

Every write path (create, update, delete on any resource) also writes an entry to `audit_log`. Rows record: actor, action, target entity + ID, timestamp, and a JSON payload snapshot. There's no delete API and no admin UI that mutates rows — the table is truly append-only, so it doubles as a forensic trail.

What we don't defend against (yet)

- **CSRF.** The API is JWT-cookie-authenticated but doesn't validate a CSRF token. If you enable cross-origin credentials in production, add CSRF protection — Spring provides it out of the box.
- **Brute force at the network layer.** Rate limiting is per-username-per-IP; a distributed brute-force from many IPs would slip through. Front the backend with a WAF or a reverse proxy with global rate limits.
- **Upload malware scanning.** Uploaded documents are text-extracted but not virus-scanned. Add ClamAV or an equivalent if you accept untrusted uploads.

Deployment.

Getting Neurix into production. The one-liner and everything you should think about after.

The one-liner

From the repository root:

```
docker compose up --build -d
```

That builds all three container images and starts them detached. You can hit `http://<host>:8082` a few seconds later.

! The `--build` flag matters. If you skip it after pulling changes, Compose will happily start containers from a stale image and you'll wonder why your bug fix isn't in production. Always rebuild after code changes.

Environment variables

The compose file reads a small set of environment variables. Set them via `.env` at the repo root or via your orchestrator's secret manager:

Variable	Purpose	Default (⚠ change in prod)
<code>JWT_SECRET</code>	Signing key for JWTs	hard-coded dev value in <code>application.yml</code>
<code>SPRING_PROFILES_ACTIVE</code>	Toggle SQLite vs. Postgres config	<code>default</code> (SQLite)
<code>SPRING_DATASOURCE_URL</code>	JDBC URL — required when switching DB	—
<code>SPRING_DATASOURCE_USERNAME</code> , <code>_PASSWORD</code>	DB credentials	—
<code>APP_CORS_ALLOWED_ORIGINS</code>	Comma-separated list of allowed origins	<code>*</code>
<code>APP_ADMIN_USERNAME</code> , <code>_PASSWORD</code>	Default admin credentials seeded on first boot	<code>admin</code> / <code>admin123</code>

Switching from SQLite to Postgres

Two files change:

1. `backend/pom.xml` — swap the SQLite dialect dependency for `postgresql`.
2. `backend/src/main/resources/application.yml` — set the JDBC URL, credentials, and dialect. Or override at runtime with env vars.

```
spring:
  datasource:
    url: jdbc:postgresql://db.internal:5432/neurix
    username: neurix
    password: ${DB_PASSWORD}
  jpa:
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

Every JPA entity uses standard column types — no SQLite-specific casts, no native queries. Hibernate will emit the correct DDL for Postgres on first boot.

Backups

With SQLite, the database file lives in the `dems-data` Docker volume. To back up:

```
docker run --rm -v dems-data:/data -v $(pwd):/backup alpine \  
tar czf /backup/dems-backup-$(date +%Y%m%d).tar.gz -C /data .
```

With Postgres, use `pg_dump` as usual. Nothing about the app assumes anything about the backup cadence — pick what fits your RPO.

Development setup.

Running the app locally without Docker, and the day-to-day workflow.

Prerequisites

- **JDK 17+** for the backend
- **Node.js 18+** and **npm** for the frontend
- Optional: **Maven 3.9+** — otherwise the `mvnw` wrapper is included

Running the backend

```
cd backend
./mvnw spring-boot:run    # macOS / Linux
mvnw.cmd spring-boot:run  # Windows
```

The API starts on `http://localhost:8080`. On first boot it creates `./data/dataentry.db` (SQLite) and seeds the default admin (`admin / admin123`) plus a sample agent (`agent1 / agent123`).

Running the frontend

```
cd frontend
npm install
npm run dev
```

Vite serves the SPA at `http://localhost:5173` with hot reload. The dev server's `proxy` setting forwards `/api/*` to the backend on port 8080, so there's no CORS setup for local dev.

Type-checking and building

```
cd frontend
npm run build    # tsc -b then vite build
```


The build is set up to fail on TypeScript errors — the `tsc -b` step runs before `vite build`. If you want a quick type-check without a full build, run `npx tsc -b --noEmit`.

The design detector hook

If you're using the Impeccable skill, the design detector runs after every edit to a UI file. It flags common anti-patterns (gradient text, oversaturated palettes, decoration for its own sake). The rules live in the skill config; per-file suppressions go in `.impeccable/config.json`.

User guides.

Task-based walkthroughs — the workflows a real person will run through in their first week on the system, written the way I'd hand them to a new teammate.

Signing in for the first time

1. Open the app in a browser: `http://<your-neurix-host>/login`.
2. The first admin account is seeded automatically on first boot. The username is `admin` and the password is `admin123` — that's a well-known default, so change it the moment you're in.
3. Type the credentials, hit *Sign in*. You'll land on the admin dashboard at `/admin`.
4. Open the user chip in the top-right, choose *Change password*, and pick something you can remember but nobody else will guess.

! If you're deploying Neurix somewhere that other people can reach, change the default admin password *before* you point DNS at the box. The default is documented in this file, in the README, and in the seed code — assume everyone knows it.

Adding a new user (admin only)

1. From the sidebar, click *Users*. You'll see the current team roster.
2. Top-right of the page, click *Add member*. A side panel slides in.
3. Fill in username, display name, role (*Admin* or *Agent*), and a starting password. Save.
4. The new user appears immediately in the roster. They can sign in with the credentials you just set.

To disable a user without deleting their history, open their row-actions menu and toggle *Active* off. Their tickets stay in the system; they just can't sign in.

Creating a department and subcategory

1. Sidebar → *Departments*. Click *Add department*.
2. Give it a name and description. Pick a color slot (1–6) — this is what appears as the colored strip on the department's card, and as the tint on its mini-stats.

3. Save. Now go to *Subcategories*, click *Add*, pick the department you just made, and give the subcategory a name.
4. The moment both exist, they'll show up as options in the ticket submission form for every agent.

✓ Subcategories are optional on a ticket, but a department is required. If you want to force agents to pick a subcategory too, add a small validation rule at `TicketService.submit()` — it's a one-line change.

Adding a custom field to the submission form

This is the killer feature. Adding a field is a UI action, not a code change — no deploy needed.

1. Sidebar → *Custom fields*. You'll see the current field list, in the order agents will see them.
2. Click *Add field*. Pick a field type (*Text*, *Number*, *Date*, *Dropdown*, *Textarea*). Set a label.
3. If you picked *Dropdown*, add options one at a time.
4. Choose whether the field is required. Save.
5. Refresh `/submit` as any agent — the new field is there, in the position you set.

To reorder fields, drag them in the field list. To disable a field without losing history, toggle *Active* off — the field disappears from the form but past ticket values remain in the database.

Submitting a ticket (as an agent)

1. Sign in as an agent (e.g. the seeded `agent1 / agent123`).
2. You'll land on your personal dashboard at `/dashboard`. Click *New ticket*, or use the sidebar to jump to `/submit`.
3. Fill in the core fields: title, department, subcategory (optional), and any custom fields the admin defined.
4. If you have a document to work from, drag it onto the upload zone. If it's a PDF or an image, the system extracts the text and pre-fills the content field.
5. Before submitting, click *Check with AI*. You'll get a cleaned-up version of your text with any suggested edits. Accept them or ignore them.
6. Hit *Submit*. The ticket appears in *My tickets* immediately and in the admin's *All tickets* view.

Running a report

1. As admin, sidebar → *Reports*.

2. At the top, pick a time range: *Today, This week, This month, All time*.
3. The page shows three cuts side-by-side: *by department, by user, by status*. Each is a chart plus a small table.
4. Click any bar or row to drill into the underlying tickets. You'll be dropped on the tickets page with the right filter pre-applied.

Using the Neurix Assistant

The floating cyan pill in the bottom-right corner is the chat assistant. Click it, ask a question, and it'll either answer or drop a button that navigates you to the right page.

Things it handles well:

- Navigation asks: *"take me to users"*, *"وديني على التقارير"*, *"open custom fields"*.
- Small talk in either language: *"hi"*, *"أهلاً"*, *"thanks"*, *"شكراً"*.
- Product questions: *"what does this app do?"*, *"الموقع يعمل ايه"*.
- Capability questions: *"what can you do?"*, *"ساعدني"*.

Conversation history persists in your browser — refresh the page and the chat picks up where you left off. To clear it, click the trash icon in the chat header.



Language detection is automatic. Ask in Arabic, get an Arabic reply; ask in English, get English. The bot decides based on the characters in your message, not your app-wide language preference.

Troubleshooting.

The issues that come up in real life and the exact fix. Ordered roughly by frequency.

The Neurix logo shows as a broken image

Symptom. You see a broken-image icon in the sidebar (or the login page) where the Neurix N should be.

Cause. Your Docker container was built *before* the logo assets were added to the frontend's `public/` directory, and the built `dist/` inside the running container doesn't have the PNGs. nginx is serving `index.html` as a fallback for those URLs, which is why you get a "200 OK" but a broken image.

Fix.

```
docker compose build frontend
docker compose up -d frontend
```

Then hard-refresh (`Ctrl+Shift+R`) so your browser doesn't reuse the cached broken response.

✓ To verify from the command line: `curl -sI http://<host>:8082/neurix-mark.png` — you want to see a `Content-Length` of ~74 KB. If it's ~500 bytes, that's the SPA fallback firing and the image really isn't in the container.

"Port already in use" when starting the dev server

Symptom. `npm run dev` prints *"Port 5173 is in use, trying another one..."* and lands on 5174, 5175, or higher.

Cause. A previous `vite` process didn't exit cleanly — usually because a Node process is still holding the port.

Fix. Either use the new port Vite fell back to, or kill the stale process:

```
# Windows PowerShell
Get-NetTCPConnection -LocalPort 5173 | ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }

# macOS / Linux
lsof -ti:5173 | xargs kill
```

Login says "Something went wrong" and you're stuck on `/login`

First check. Open the browser dev tools → Network tab, submit again, and look at `POST /api/auth/login`. What HTTP status came back?

- **401.** Wrong credentials. The default admin is `admin / admin123`; the default agent is `agent1 / agent123`.
- **403.** Your account is disabled (`Active = false`). Ask an admin to re-enable you.
- **429.** The rate limiter has kicked in — too many failed attempts. Wait 5 minutes.
- **0 / CORS error.** The backend isn't reachable. In Docker, `docker compose ps`; if `dems-backend` isn't *Up*, look at `docker compose logs backend`.
- **500.** The backend crashed. `docker compose logs backend --tail=50` and read the last few stack traces.

Chat says "I don't quite understand" for almost everything I ask

Cause. The current chat backend is rule-based keyword matching. It handles a fixed vocabulary well but stumbles on paraphrases, typos, and long compound questions.

Workaround. Stick to short, direct asks: *"open users"*, *"وديني على التقارير"*, *"what does this do?"*.

Real fix. Swap `ChatService.reply()` in the backend for a real LLM call. The response contract (`{ reply, actions }`) doesn't change — the frontend picks up the improved understanding automatically. See Chapter 14 for the roadmap on this.

JWT expired mid-session, all API calls return 401

Cause. Tokens expire after 24 hours. The frontend catches the 401 and bounces you to `/login`, so this shouldn't be user-facing — but if you're calling the API directly (with `curl` or Postman), you'll see it.

Fix. Re-authenticate: `POST /api/auth/login` with your credentials. Grab the new token from the response body.

"CORS policy: No 'Access-Control-Allow-Origin' header"

Cause. The frontend is calling the backend from a different origin than what `APP_CORS_ALLOWED_ORIGINS` permits. Default is `*`, which works everywhere — if you tightened it in production, make sure the actual origin is in the list.

Fix. Update `APP_CORS_ALLOWED_ORIGINS` in your compose file's `backend` service environment and restart:

```
APP_CORS_ALLOWED_ORIGINS=https://neurix.yourcompany.com
```

Multiple origins are comma-separated. Restart the backend container after changing.

Database migration on Postgres fails on first boot

Cause. Hibernate is trying to create tables against a schema it doesn't have permission for, or the Postgres user isn't the owner of the target database.

Fix. Make sure the DB user in `SPRING_DATASOURCE_USERNAME` is the owner of the database (`ALTER DATABASE neurix OWNER TO neurix;`) and has `CREATE` on the schema (`GRANT ALL ON SCHEMA public TO neurix;`).

PDF extraction returns empty text for scanned PDFs

Cause. The text-based extractor (`PdfExtractionService`) doesn't run OCR — it only pulls text from PDFs that have real text layers. Scanned PDFs are just images and return an empty string.

Fix. Those uploads route through `PdfOcrService` instead. If that service isn't returning text either, check that its OCR backend (Tesseract or the configured cloud service) is reachable and has the right language packs installed.

The frontend build succeeds but the deployed app is blank

Cause. Almost always: nginx is serving an old `index.html` that references JavaScript bundle names from a previous build, and those bundle files no longer exist.

Fix. Rebuild and redeploy in one shot: `docker compose build frontend && docker compose up -d frontend` . Don't skip the `--build` or you'll deploy the old image again.

Roadmap & known limitations.

What's stubbed, what's on the list, and what we're deliberately not building.

Stubbed, will be replaced

- **AI content check.** `AiCheckService` does deterministic cleanup today. The API shape is already the shape a real LLM would return — the swap is a single-method change.
- **Neurix Assistant intent classification.** Rule-based keyword matching today. Migrating to an LLM would improve fuzzy understanding (typos, indirect asks) but the current implementation is deliberately good enough to ship.

On the near-term list

- **Soft deletes for users and tickets.** Hard deletes today; audit log preserves history but the row is gone.
- **Bulk import.** Uploading a CSV and mapping columns to fields. A common request from ops.
- **Email notifications.** On status changes and daily digests for admins.
- **SSO.** SAML or OIDC integration for enterprise deployments.
- **Test coverage.** There isn't much yet. This is the single biggest debt in the codebase.

Deliberately not on the list

- **Real-time collaboration.** Two admins editing the same department at the same time is a rare enough scenario that last-write-wins is fine. Adding CRDTs or OT would be overkill.
- **Mobile app.** The web app is fully responsive; a native app doesn't buy enough to justify the maintenance.
- **Microservices.** The system is small enough to stay a monolith. Splitting the audit log or the reporting service out is a decision to make when the monolith actually hurts, not before.

File layout.

The directories you'll open the most, and what they hold.

```

data_entry/
├── backend/
│   ├── pom.xml                Maven build, Spring Boot parent
│   ├── Dockerfile            Multi-stage: builder → JRE
│   └── src/main/
│       ├── java/com/dataentry/
│           ├── DataEntryApplication.java    entry point
│           ├── config/                     SecurityConfig, CorsConfig, ...
│           ├── controller/                 12 REST controllers
│           ├── service/                    20 Spring services
│           ├── model/                      13 JPA entities + enums
│           ├── repository/                 Spring Data interfaces
│           ├── dto/                       Request/response records
│           └── security/                   JWT filter, user details
│       └── resources/
│           ├── application.yml             config: DB, CORS, JWT
│           └── db/                        seed data, migrations
├── frontend/
│   ├── package.json            React 18, TS 5, Vite 5
│   ├── vite.config.ts          dev proxy for /api → :8080
│   ├── nginx.conf              prod SPA fallback + API proxy
│   ├── Dockerfile              build → nginx serve
│   ├── public/
│       ├── neurix-logo.png      full logo, dark wordmark
│       ├── neurix-logo-light.png full logo, white wordmark
│       └── neurix-mark.png       the N mark alone
│   └── src/
│       ├── App.tsx              route tree
│       ├── main.tsx             mounts <App /> with providers
│       ├── api/                 axios wrappers per resource
│       ├── components/          Layout, Modal, chat/, auth/, ...
│       ├── context/             Auth, Theme, Preferences
│       ├── i18n/                 en.ts + ar.ts dictionaries
│       ├── pages/               24 pages (admin/, user/, login)
│       └── styles/global.css     the design system, 2.5k lines
└── docker-compose.yml           3 services: frontend, backend, translate

```

A few file-name conventions to know

- **Every REST controller is** `<Resource>Controller.java` and pairs 1:1 with a `<Resource>Service.java`. If you're looking for the code behind an endpoint, start at the controller and follow one hop.
- **Frontend API wrappers** mirror the backend: `src/api/tickets.ts` wraps `TicketController`, `src/api/chat.ts` wraps `ChatController`. Same shape, same names.
- **Admin-only pages live under** `src/pages/admin/` and are gated by `<ProtectedRoute roles={['ADMIN']}>` in `App.tsx`. User pages under `src/pages/user/` are accessible to both roles.
- **DTOs are Java records** defined inside a `<Resource>Dtos.java` class, grouped as nested records. This keeps request/response types adjacent without spawning a file per DTO.

Glossary.

Short definitions for the terms that show up all over this document.

Term	Meaning
Agent	A user with the <code>USER</code> role. Fills in tickets, sees their own dashboard and their own submissions.
Admin	A user with the <code>ROLE_ADMIN</code> role. Can access every page an agent can, plus admin-only screens.
Audit log	Append-only history of every write action in the system. Backed by the <code>audit_log</code> table; no delete API.
BCrypt	The password-hashing algorithm the app uses. Cost factor 10 (Spring default).
Callout	A boxed note in the UI or in this document. Three variants: neutral, tip (green), warning (amber).
CORS	Cross-Origin Resource Sharing. Controls which browser origins the backend accepts requests from. Configured via <code>APP_CORS_ALLOWED_ORIGINS</code> .
Custom field	An admin-defined field that appears on the ticket submission form. Type-typed (text / number / date / dropdown / textarea).
Department	Top level of the ticket taxonomy. Every ticket is filed against one.
DTO	Data Transfer Object. The Java records in <code>dto/</code> that shape request and response payloads.
EAV	Entity–Attribute–Value pattern. Used by <code>TicketFieldValue</code> to store custom-field values without adding columns.
Focus ring	The cyan halo that appears around interactive elements when they hold keyboard focus. WCAG-required.
JPA	Jakarta Persistence API. The ORM abstraction Spring Data uses to talk to the database (backed by Hibernate here).
JWT	JSON Web Token. The signed credential the backend issues on successful login and validates on every subsequent request.
KPI	Key Performance Indicator. The big stat cards at the top of the dashboards.
Neurix Assistant	The chat widget in the bottom-right corner. Bilingual, navigation-aware, rule-based (LLM-ready).
nginx	The reverse proxy in front of the frontend container. Serves the SPA and proxies <code>/api/*</code> to the backend.

Term	Meaning
Pill	A rounded (radius 999px) shape used for buttons and status badges. Consistent shape across the whole app.
Project	An orthogonal grouping over tickets, used mainly for reporting. A project can span departments.
ProtectedRoute	A React component that guards a route by requiring a signed-in user (and optionally a specific role).
RBAC	Role-Based Access Control. Neurix has two roles: <code>ADMIN</code> and <code>USER</code> .
Rate limiter	The (IP, username) throttle on failed login attempts. In-memory by default; DB-backed variant available.
Skeleton	The soft gray shape the UI shows while content is loading — used instead of a spinner for perceived speed.
SPA	Single-Page Application. What React renders in the browser. nginx serves <code>index.html</code> for all unknown paths so client-side routing works on refresh.
Subcategory	An optional second level under a department. Refines the ticket taxonomy.
Ticket	A single data-entry submission. The core write-heavy record in the system.
Toast	A brief, dismissible notification that slides in from a corner of the screen.
Token	Two meanings in this doc: (1) a JWT credential; (2) a design token (a CSS variable like <code>--brand</code>).
WCAG	Web Content Accessibility Guidelines. Neurix targets WCAG AA for color contrast and focus indication.